Program 1: Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup

Overview of Build Automation Tools

Build automation tools help developers streamline the process of building, testing, and deploying software projects. They take care of repetitive tasks like compiling code, managing dependencies, and packaging applications, which makes development more efficient and error-free.

Two popular tools in the Java ecosystem are **Maven** and **Gradle**. Both are great for managing project builds and dependencies, but they have some key differences.

Maven

- Maven is a build automation tool primarily used for Java projects. It uses an XML configuration file called `pom.xml` (Project Object Model) to define project settings, dependencies, and build steps.
- **Main Features:**
  - Predefined project structure and lifecycle phases.
  - Automatic dependency management through Maven Central.
  - Wide range of plugins for things like testing and deployment.
  - Supports complex projects with multiple modules.

Gradle

- Gradle is a more modern and versatile build tool that supports multiple programming languages, including Java, Groovy, and Kotlin. It uses a domain-specific language (DSL) for build scripts, written in Groovy or Kotlin.
- **Main Features:**
  - Faster builds thanks to task caching and incremental builds.
  - Flexible and customizable build scripts.
  - Works with Maven repositories for dependency management.
  - Excellent support for multi-module and cross-language projects.
- Integrates easily with CI/CD pipelines.

Key Differences Between Maven and Gradle

Aspect	Maven	Gradle
Configuration	XML (`pom.xml`)	Groovy or Kotlin DSL
Performance	Slower	Faster due to caching
Flexibility	Less flexible	Highly customizable
Learning Curve	Easier to pick up	Slightly steeper
Script Size	Verbose	More concise
Dependency Management	Uses Maven Central	Compatible with Maven too
Plugin Support	Large ecosystem	Extensible and versatile

Installation and Setup

Java (JDK) Installation:

Step 1: Type the command **sudo apt install openjdk-11-jdk -y in the terminal**

It will install the OpenJDK 17 (Java Development Kit) and accepts installation automatically.

```
varun-c-m@varun-c-m-VirtualBox:~$ sudo apt install openjdk-11-jdk -y
[sudo] password for varun-c-m:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openjdk-11-jdk is already the newest version (11.0.30+7-1ubuntu1~24.04).
0 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
```

The installation will start like this.

Step 2: Installation can be confirmed by running the command **java -version**

```
varun-c-m@varun-c-m-VirtualBox:~$ java -version
openjdk version "17.0.18" 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-124.04.1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-124.04.1, mixed mode, sharing)
```

Maven Installation:

Step 1: Type the command **sudo apt install maven** in the terminal.

It will install Apache Maven from Ubuntu's package repository.

```
varun-c-m@varun-c-m-VirtualBox:~$ sudo apt install maven
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
maven is already the newest version (3.8.7-2).
0 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
```

Step 2: Installation can be confirmed by running the command **mvn -version**

```
varun-c-m@varun-c-m-VirtualBox:~$ mvn -version
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 17.0.18, vendor: Ubuntu, runtime: /usr/lib/jvm/java-17-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "6.17.0-14-generic", arch: "amd64", family: "unix"
```

Gradle Installation:

Step 1: Type the command **sudo apt install gradle** in the terminal

It installs Gradle build automation tool from Ubuntu's package repository.

```
varun-c-m@varun-c-m-VirtualBox:~$ sudo apt install gradle
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gradle is already the newest version (4.4.1-20).
0 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
varun-c-m@varun-c-m-VirtualBox:~$
```

Step 2: To check the proper installation of Gradle run the command **gradle –version**

```
varun-c-m@varun-c-m-VirtualBox:~$ gradle --version
openjdk version "17.0.18" 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-124.04.1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-124.04.1, mixed mode, sharing)

------------------------------------------------------------
Gradle 4.4.1
------------------------------------------------------------

Build time:   2012-12-21 00:00:00 UTC
Revision:     none

Groovy:       2.4.21
Ant:          Apache Ant(TM) version 1.10.14 compiled on September 25 2023
JVM:          17.0.18 (Ubuntu 17.0.18+8-Ubuntu-124.04.1)
OS:           Linux 6.17.0-14-generic amd64
```

Program 2: Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

Step 1: To create a Maven project run the following command in terminal:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

This command generates a basic Maven project structure with a sample App.java and AppTest.java.

```
varun-c-m@varun-c-m-VirtualBox:~$ mvn archetype:generate -DgroupId=com.example -Dartifa
ctId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=fals
e
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ sta
ndalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ sta
ndalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18
 MB at 23 MB/s)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-
archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/varun-c-m
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/varun-c-m/MyMavenApp
[INFO] ----------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------------
```

This "BUILD SUCCESS" will confirm that the project is created successfully

```
varun-c-m@varun-c-m-VirtualBox:~$ cd devops6d
varun-c-m@varun-c-m-VirtualBox:~/devops6d$ cd MyMavenApp
varun-c-m@varun-c-m-VirtualBox:~/devops6d/MyMavenApp$ ls
pom.xml  src  target
```

Step 2: Go to the project directory using command **cd MyMavenApp**

Step 3: Open and edit pom.xml file using command **gedit pom.xml**

- Go to https://mvnrepository.com
- Search for a dependency "junit"
- Select the latest version and copy the dependency XML snippet.
- Paste it under the <dependencies> section in pom.xml
- Updated pom.xml:

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

```
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>MyMavenApp</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>
<name>MyMavenApp</name><url>http://maven.apache.org</url>
<!-- Java version configuration -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
<!-- Dependencies -->
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
</dependencies>
<!-- Build Plugins -->
<build>
<plugins>
<!-- Compiler Plugin -->
<plugin>
<groupId>org.apache.maven.plugins</groupId> <artifactId>maven-compiler-
plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>11</source>
<target>11</target>
</configuration>
</plugin>
<!-- Surefire Plugin for Testing -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<!-- JAR Plugin with Entry Point -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<version>3.1.0</version>
```

```
<configuration>
<archive>
<manifestEntries>
<Main-Class>com.example.App</Main-Class>
</manifestEntries>
</archive
```

```
</configuration>
  </plugin>
  </plugins>
  </build>
  </project>
```

Step 4: Build the project using command **mvn clean install**

The “Build Success” message ensures that our Maven project is built successfully.

Step 5: To run the Maven application type he command:

java -jar target/MyMavenApp-1.0-SNAPSHOT.jar

- After completing the execution, the output will look like:

This will show output as the code present in App.java file we can edit the text to get any other output.

Creating a Maven Project with Guava and Commons-IO

Google Guava Dependency

Guava is a set of core libraries developed by Google to fill gaps in Java's standard libraries. It is used for:

- Collections enhancements: Multimap, BiMap, Table, ImmutableList, etc.
- Functional programming utilities: Predicates, Functions, Suppliers.
- Caching: Built-in caching mechanisms (CacheBuilder) like a mini in-memory cache.String utilities: Splitter, Joiner, CharMatcher.
- Concurrency: ListenableFuture, Service, and other helper classes.
- Hashing and hashing strategies: Useful for consistent hashing (e.g., Bloom filters). Overall, Guava simplifies and improves many core Java patterns and utilities.

Apache Commons IO Dependency

Apache Commons IO is focused purely on input/output utilities, especially for file and stream handling. It is used for:

- File operations: Easily read/write files with FileUtils.
- Stream utilities: IOUtils.copy(), IOUtils.toString() — simplifies stream handling.
- Directory monitoring: FileAlterationObserver for detecting changes in a folder.
- File filtering: WildcardFileFilter, SuffixFileFilter.

Overall, Commons IO reduces boilerplate and errors when working with files or streams.

Maven Project with Guava and Commons-IO Dependencies

Create a Maven Project Using Terminal by following command:

mvn archetype: generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
varun-c-m@varun-c-m-VirtualBox:~$ mvn archetype:generate -DgroupId=com.example -Dartifa
ctId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode
=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ sta
ndalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ sta
ndalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-
archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/varun-c-m
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenGuavaApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/varun-c-m/MyMavenGuavaApp
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.701 s
[INFO] Finished at: 2026-03-24T23:50:01+05:30
```

Now Navigate to the Project Directory using **cd MyMavenGuavaApp**

Edit the pom.xml File and add the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
    v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId><artifactId>MyMavenGuavaApp</artifactId>
      <packaging>jar</packaging>
      <version>1.0-SNAPSHOT</version>
      <name>MyMavenGuavaApp</name>
      <url>http://maven.apache.org</url>
      <dependencies>
        <dependency>
          <groupId>junit</groupId>
          <artifactId>junit</artifactId>
          <version>3.8.1</version>
          <scope>test</scope>
```

```
    </dependency>

    <!-- https://mvnrepository.com/artifact/com.google.guava/guava -->
<dependency>
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>33.4.0-jre</version>
</dependency>

<!-- https://mvnrepository.com/artifact/commons-io/commons-io -->
<dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.18.0</version>
</dependency>

  </dependencies>

  <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>

<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
            <archive>
                <manifestEntries>
                    <Main-Class>com.example.App</Main-Class>
                </manifestEntries>
            </archive>
```

```
        </configuration>
    </plugin>

</plugins>
</build>
</project>
```

Modify App.java

Got to App.java using **cd src/main/java/com/example/** and **gedit App.java** to type the file with following code:

```
package com.example;

/**
 * Hello world!
 *
 */
// Import FileUtils class from Apache Commons IO library
// This is used to perform file operations like copying files
import org.apache.commons.io.FileUtils;

// Import File class to represent file and directory pathnames
import java.io.File;

// Import IOException to handle input-output errors
import java.io.IOException;

// Import ImmutableList from Google Guava library
// ImmutableList creates a list that cannot be modified after creation
//import com.google.common.collect.ImmutableList;

// Define the main class
public class App
{
    // Main method - program execution starts from here
    public static void main( String[] args )
    {
        // Create an immutable list of strings (fruits)
        // This list cannot be changed (no add/remove allowed)
```

```
        ImmutableList<String> fruits = ImmutableList.of("Apple", "Banana", "Cherry");

        // Print the list of fruits to the console
        System.out.println(fruits);

        // Create a File object representing the source file
        File sourceFile = new File("source.txt");

        // Create a File object representing the destination file
        File destFile = new File("destination.txt");

        try {
            // Copy the contents of source.txt to destination.txt
            // Using Apache Commons IO FileUtils class
            FileUtils.copyFile(sourceFile, destFile);

            // Print success message if file is copied
            System.out.println("File copied successfully!");
        }
        catch (IOException e) {
            // If any error occurs during file copying,
            // this block will handle the exception
            // Print error message
            System.err.println("Error occurred while copying file: " + e.getMessage());
        }
    }
}
```

Build the Project using the command **mvn clean install** command

```
varun-c-m@varun-c-m-VirtualBox:~/devops/MyMavenGuavaApp$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.example:MyMavenGuavaApp >-------------------
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenGuavaApp ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/varun-c-m/devops/MyMavenGuavaApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenGuavaApp ---
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to /home/varun-c-m/devops/MyMavenGuavaApp/target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/varun-c-m/devops/MyMavenGuavaApp/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenGuavaApp ---
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to /home/varun-c-m/devops/MyMavenGuavaApp/target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenGuavaApp ---
[INFO] -------------------------------------------------------
```

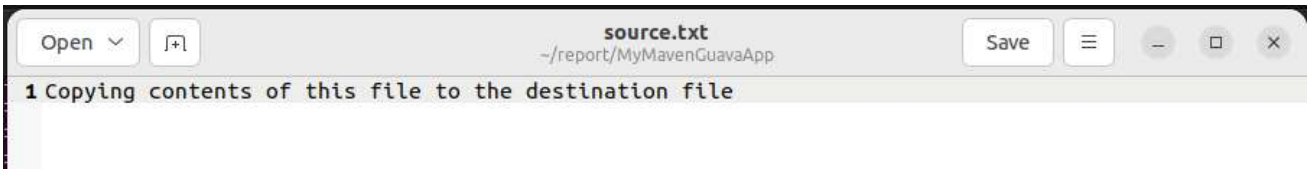
This compiles the project and creates a JAR in the target/ folder.

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.059 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenGuavaApp ---
[INFO] Building jar: /home/vboxuser/devops/MyMavenGuavaApp/target/MyMavenGuavaApp-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenGuavaApp ---
[INFO] Installing /home/vboxuser/devops/MyMavenGuavaApp/target/MyMavenGuavaApp-1.0-SNAPSHOT.jar to /home/vboxuser/.m2/re
App/1.0-SNAPSHOT/MyMavenGuavaApp-1.0-SNAPSHOT.jar
[INFO] Installing /home/vboxuser/devops/MyMavenGuavaApp/pom.xml to /home/vboxuser/.m2/repository/com/example/MyMavenGuav
-1.0-SNAPSHOT.pom
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  10.495 s
[INFO] Finished at: 2026-03-21T10:02:34+05:30
[INFO] ------------------------------------------------------------------------
```

Run the Application:

If java-jar command give an error which is most probable, use the following command
mvn exec:java -Dexec.mainClass=”com.example.App”
This command gives error asError while copying file:Source ‘source.txt’ does not exist

Now, Create a Sample File to copy the content to destination file



Now Run the command **mvn exec:java -Dexec.mainClass=”com.example.App”** again

```
varun-c-m@varun-c-m-VirtualBox:~/devops/MyMavenGuavaApp$ mvn exec:java -Dexec.mainClass=com.example.A
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.example:MyMavenGuavaApp >-------------------
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec-maven-plugin:3.6.3:java (default-cli) @ MyMavenGuavaApp ---
Error occurred while copying file: Source 'source.txt' does not exist
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.704 s
[INFO] Finished at: 2026-03-21T11:25:10+05:30
[INFO] ------------------------------------------------------------------------
```

The output shows file copied successfully.
We can see newly created **destination.txt** using gedit destination.txt if you are still in Project’s directory

```
varun-c-m@varun-c-m-VirtualBox:~/devops/MyMavenGuavaApp$ ls
destination.txt  pom.xml  source.txt  src  target
varun-c-m@varun-c-m-VirtualBox:~/devops/MyMavenGuavaApp$
```

File content is same as the content in source file.

Program 3: Working with Selenium: Maven Project To Connect To Web Servers Using Selenium Dependency

1. Sauce.demo webpage

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false**

```
varun-c-m@varun-c-m-VirtualBox:~/devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -Darchetyp
[INFO] Scanning for projects...
[INFO] --------------------< org.apache.maven:standalone-pom >--------------------
[INFO] Building Maven Stub Project (No POM)
[INFO] ---------------------------[ pom ]----------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[WARNING] ----------------------------------------------------------------------------
[WARNING] Using following parameters for generating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[WARNING] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/varun-c-m/devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenSeleniumApp01
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/varun-c-m/devops/MyMavenSeleniumApp01
[INFO] BUILD SUCCESS
[INFO] Total time:  3.164 s
[INFO] Finished at: 2026-03-21T11:33:28+05:30
```

Step 2: Before we connect to web servers we need to have chrome driver installed. To download chrome driver

To check the chrome version type the command **google-chrome –version** in the terminal

extract the version and store it in a variable CHROME_VERSION

extracts and transorms JSON data

LATEST_DRIVER=$(curl -s "https://googlechromelabs.github.io/chrome-for-testing/latest-patch-versions-per-build.json" | jq -r ".builds[\"$CHROME_VERSION\"].version")

do install curl using the command

sudo apt install curl

wget https://storage.googleapis.com/chrome-for-testing-public/$LATEST_DRIVER/linux64/chromedriver-linux64.zip

Extract and move it to /usr/local/bin/:

unzip chromedriver-linux64.zip

sudo mv chromedriver-linux64/chromedriver /usr/local/bin/chromedriver

sudo chmod +x /usr/local/bin/chromedriver

```
varun-c-m@varun-c-m-VirtualBox:~$ chromedriver --version
ChromeDriver 145.0.7632.117 (de4e5d6e13b995f5e04137970c0e8ec456017467-refs/branch-heads
/7632@{#3101})
```

Check if ChromeDriver is installed correctly:

Step 3 : edit the pom.xml file and add the dependencies
The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenSeleniumApp01</artifactId>
  <packaging>jar</packagi<ng>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenSeleniumApp01</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  <!--  https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java  -->
<dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.29.0</version>
</dependency>
  </dependencies>
  <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
          <groupId>org.apache.maven.plugins</groupId>
```

```
            <artifactId>maven-jar-plugin</artifactId>
            <version>3.0.1</version>
           <configuration>
               <archive>
                   <manifestEntries>
                       <Main-Class>com.example.App</Main-Class>
                   </manifestEntries>
               </archive>
            </configuration>

        </plugin>

</plugins>
</build>
</project>
```

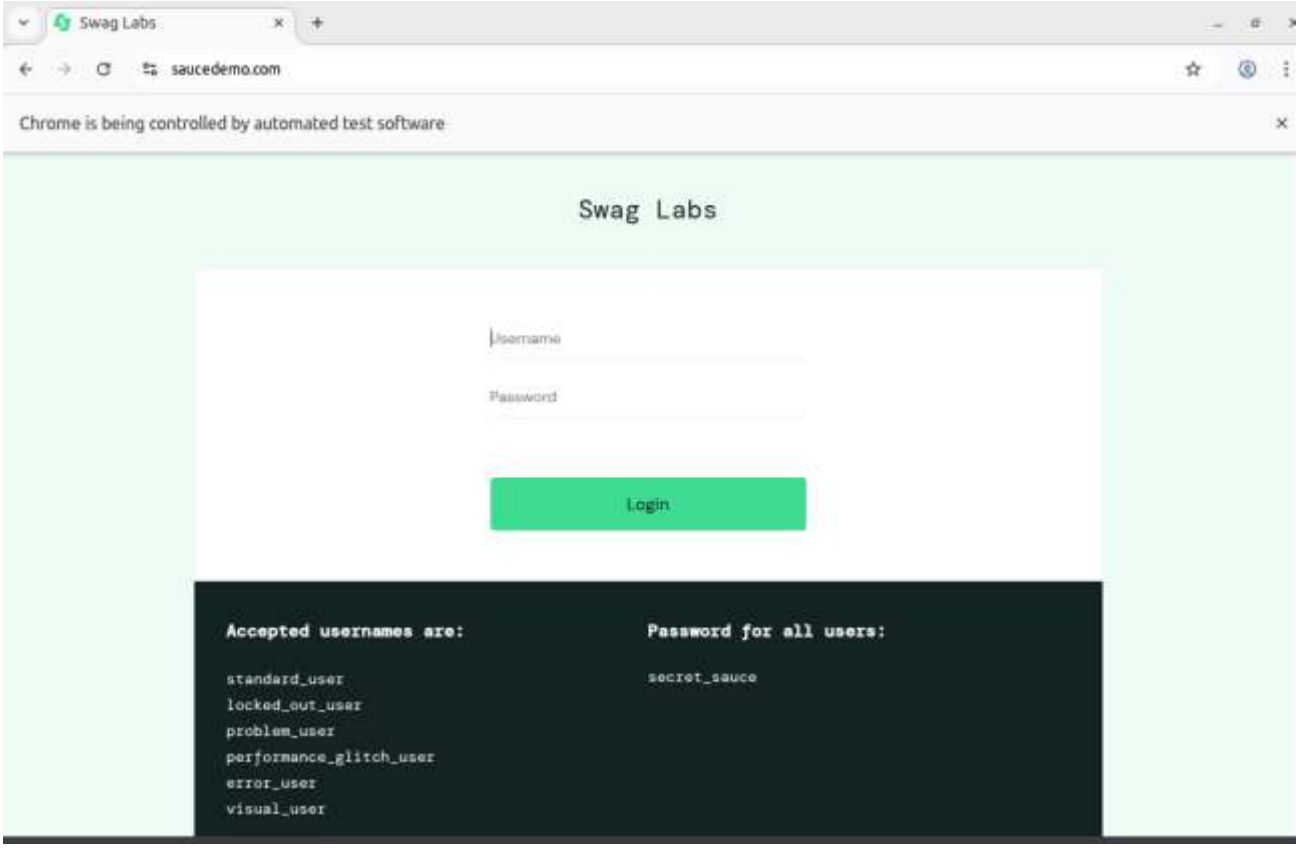
Step 4: Edit the App.java

```
package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

/**
 * Hello world!
 *
 */
public class App
{
    public static void main( String[] args )
    {
        WebDriver driver=new ChromeDriver();
        driver.get("https://www.saucedemo.com/");
        driver.manage().window().maximize();
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();
    }
}
```

```
varun-c-m@varun-c-m-VirtualBox:~/devops/MyMavenSeleniumApp01$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.example:MyMavenSeleniumApp01 >--------------------
[INFO] Building MyMavenSeleniumApp01 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenSeleniumApp01 ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenSeleniumApp01 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/varun-c-m/devops/MyMavenSeleniumApp01/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenSeleniumApp01 ---
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to /home/varun-c-m/devops/MyMavenSeleniumApp01/target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenSeleniumApp01 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/varun-c-m/devops/MyMavenSeleniumApp01/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenSeleniumApp01 ---
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to /home/varun-c-m/devops/MyMavenSeleniumApp01/target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenSeleniumApp01 ---
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
```

```
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- maven-jar-plugin:3.0.1:jar (default-jar) @ MyMavenSeleniumApp01 ---
[INFO] Building jar: /home/vboxuser/devops/MyMavenSeleniumApp01/target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenSeleniumApp01 ---
[INFO] Installing /home/vboxuser/devops/MyMavenSeleniumApp01/target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar to /home/vboxuser/.m2/repository/com/exam
MavenSeleniumApp01/1.0-SNAPSHOT/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar
[INFO] Installing /home/vboxuser/devops/MyMavenSeleniumApp01/pom.xml to /home/vboxuser/.m2/repository/com/example/MyMavenSeleniumApp01/1.0-SNAPSHOT
enSeleniumApp01-1.0-SNAPSHOT.pom
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  7.893 s
```

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**
it opens saucedemo.com and we can login using the credentials



2. practiceautomation.com

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenPracticeAutomation -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false**

```
varun-c-m@varun-c-m-VirtualBox:~/devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenPracticeAutomation -Darchetyp
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------[ pom ]---------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] ----------------------------------------------------------------------------
[WARNING] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/varun-c-m/devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenPracticeAutomation
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/varun-c-m/devops/MyMavenPracticeAutomation
[INFO] ----------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------------
[INFO] Total time:  3.581 s
[INFO] Finished at: 2026-03-21T12:24:19+05:30
[INFO] ----------------------------------------------------------------------------
varun-c-m@varun-c-m-VirtualBox:~/devops$
```

Step 2 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.example</groupId>
 <artifactId>MyMavenPracticeAutomationApp</artifactId>
 <packaging>jar</packaging>
 <version>1.0-SNAPSHOT</version>
 <name>MyMavenPracticeAutomationApp</name>
 <url>http://maven.apache.org</url>


 <dependencies>
   <dependency>
     <groupId>junit</groupId>
     <artifactId>junit</artifactId>
     <version>3.8.1</version>
     <scope>test</scope>
   </dependency>


   <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
<dependency>
   <groupId>org.seleniumhq.selenium</groupId>
   <artifactId>selenium-java</artifactId>
   <version>4.29.0</version>
</dependency>
 </dependencies>

 <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
            <groupId>org.apache.maven.plugins</groupId>

            <artifactId>maven-jar-plugin</artifactId>
            <version>3.0.1</version>
           <configuration>
                <archive>
                    <manifestEntries>
                        <Main-Class>com.example.App</Main-Class>
                    </manifestEntries>
                </archive>
            </configuration>

        </plugin>


</plugins>
</build>
</project>
```

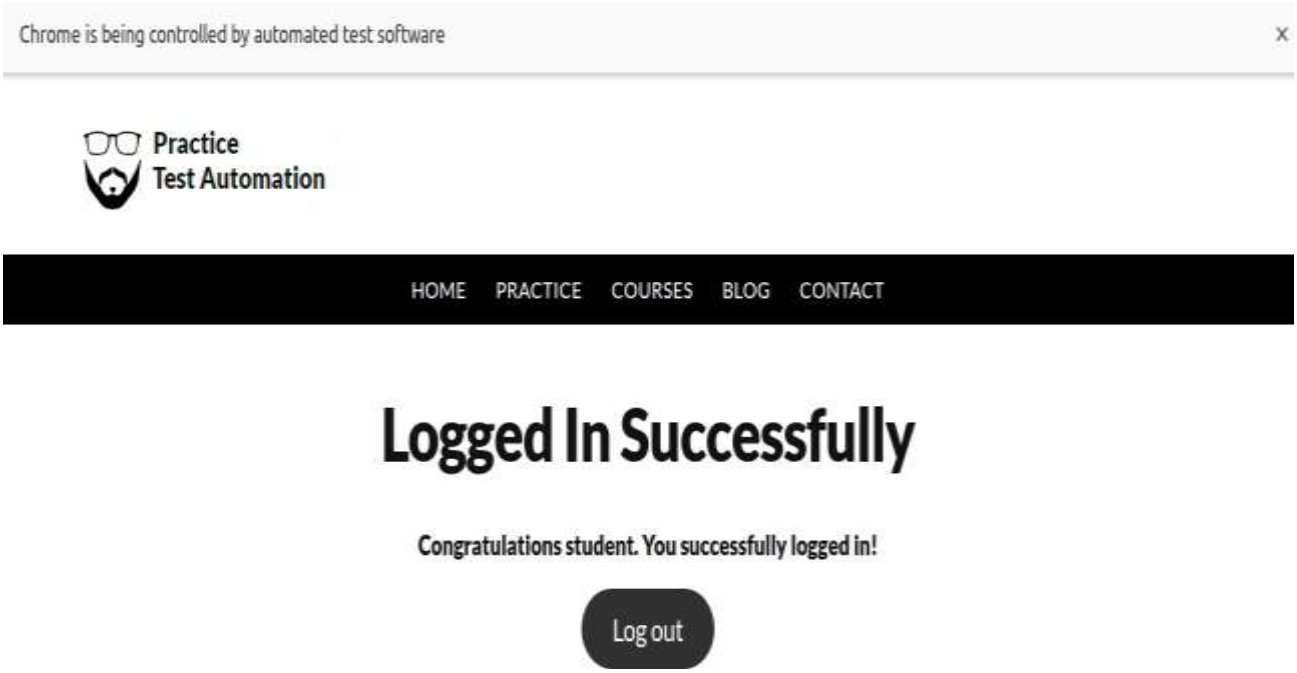
Step 3: Edit the App.java

```
package com.example;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class App
{
    public static void main( String[] args )
    {
        WebDriver driver = new ChromeDriver();
        driver.get("https://practicetestautomation.com/practice-test-login/");
        driver.manage().window().maximize();
        driver.findElement(By.id("username")).sendKeys("student");
        driver.findElement(By.id("password")).sendKeys("Password123");
        driver.findElement(By.id("submit")).click();
    }
}
```

Step 4:build and run the maven application

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**

it openspracticeautomation.com and we can login using the credentials



3. automationexercise.com

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenAutomationExercise -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false**

```
varun-c-m@varun-c-m-VirtualBox:~/devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenAutomationExercise -Darchetyp
[INFO] Scanning for projects...
[INFO]
[INFO] --------------------< org.apache.maven:standalone-pom >--------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] ----------------------------------------------------------------------------
[WARNING] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/varun-c-m/devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenAutomationExercise
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/varun-c-m/devops/MyMavenAutomationExercise
[INFO] ----------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------------
[INFO] Total time:  4.903 s
[INFO] Finished at: 2026-03-21T12:40:22+05:30
[INFO] ----------------------------------------------------------------------------
varun-c-m@varun-c-m-VirtualBox:~/devops$
```

Step 2 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">

   <modelVersion>4.0.0</modelVersion>
   <groupId>com.example</groupId>
   <artifactId>MyMavenExerciseAutomationApp</artifactId>
   <packaging>jar</packaging>
   <version>1.0-SNAPSHOT</version>
   <name>MyMavenExerciseAutomationApp</name>
   <url>http://maven.apache.org</url>

   <dependencies>
      <dependency>
         <groupId>junit</groupId>
         <artifactId>junit</artifactId>
```

```
            <version>3.8.1</version>
            <scope>test</scope>
        </dependency>
        <!--  https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java  -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.29.0</version>
        </dependency>

        <dependency>
            <groupId>io.github.bonigarcia</groupId>
            <artifactId>webdrivermanager</artifactId>
            <version>5.8.0</version>
        </dependency>
    </dependencies>
<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
            <groupId>org.apache.maven.plugins</groupId>

            <artifactId>maven-jar-plugin</artifactId>
            <version>3.0.1</version>
           <configuration>
                <archive>
                    <manifestEntries>
                        <Main-Class>com.example.App</Main-Class>
                    </manifestEntries>
                </archive>
```

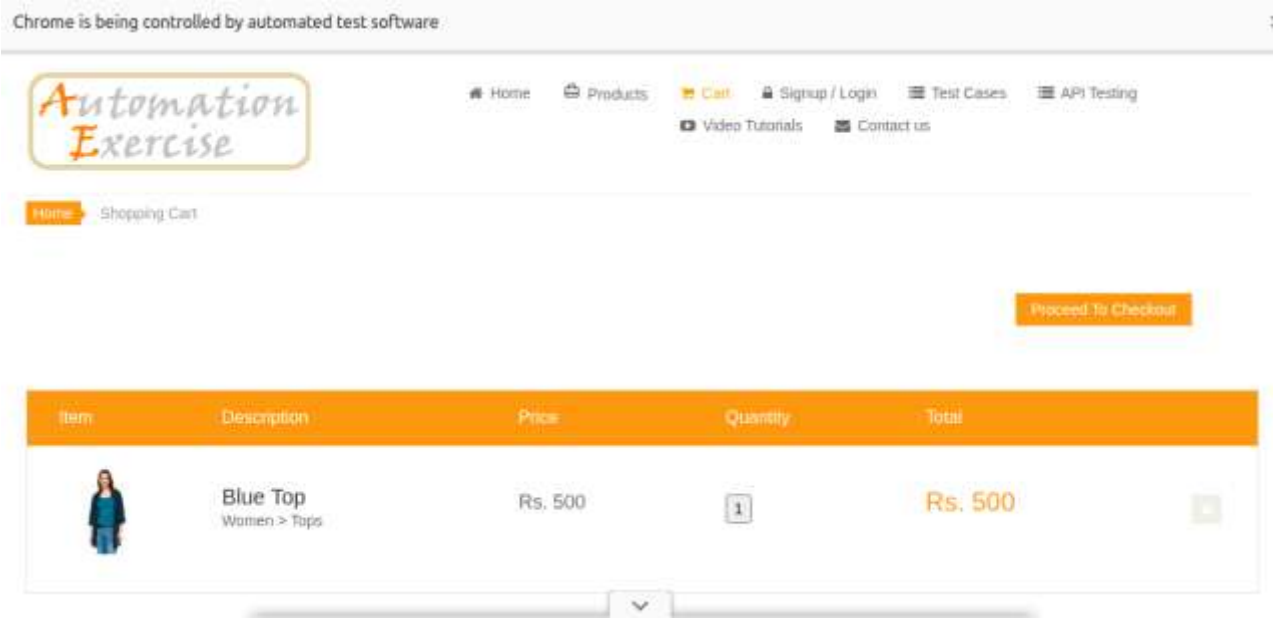
```
            </configuration>
        </plugin>
</plugins>
</build>
</project>
```

Step 3: Edit the App.java

```
package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;

import io.github.bonigarcia.wdm.WebDriverManager;
import java.time.Duration;

public class App {
    public static void main(String[] args) {

        WebDriverManager.chromedriver().setup();
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();

        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

        try {
            driver.get("https://automationexercise.com");

            // Click Products page

wait.until(ExpectedConditions.elementToBeClickable(By.xpath("//a[@href='/products']"))).clic
k();

            // Find first Add to Cart button
            WebElement addToCart = wait.until(ExpectedConditions.elementToBeClickable(
                    By.xpath("//a[@data-product-id='1']")));

            // Scroll and click using JavaScript (avoids ad overlay)
            ((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);",
addToCart);
            ((JavascriptExecutor)  driver).executeScript("arguments[0].click();",  addToCart);
```

```
            // Wait for confirmation
            modalwait.until(ExpectedConditions.visibilityOfElementLocated(By.id("cartModal")));

            // Click View Cart
            wait.until(ExpectedConditions.elementToBeClickable(By.linkText("View
Cart"))).click();

            System.out.println("Product successfully added to cart!");

        } catch (Exception e) {
            System.out.println("TEST FAILED: " + e.getMessage());
        }

        System.out.println("Browser will remain open for inspection.");
    }
}
```

Step 4:build and run the maven application

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**

it opens automationexercise.com and automatically adds a product to cart